

Correction des exercices

Chapitre 2

Exercice n°1

```
package exercice1;
class Point {
    private double x,y;
    void initialise() {
        x = 0;
        y = 0;
    }
    void deplace() {
        x += 2;
        y += 2;
    }
    void affiche() {
        System.out.println("Abscisse: " +x+ " Ordonnee: "+y);
    }
}
public class TestPoint {
    public static void main(String[] args) {
        Point p = new Point();
        p.initialise();
        p.affiche();
        p.deplace();
        p.affiche();
        p.deplace();
        p.affiche();
    }
}
```

Exercice n°2

```
package exercice2;
class Point {
    private double x;
    private double y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public Point(double c) {
        x=y=c;
    }

    void affiche() {
```

```

        System.out.println("X= "+x+ " Y= "+y);
    }
    double getX() { return x; }
    double getY() { return y; }
}
public class Principale {
    public static void main(String[] args) {
        Point p = new Point(2,3);
        Point q = new Point(-2,-3);
        if( (p.getY() == q.getY() ) && (p.getX() == -q.getX() ) ) {
            System.out.println("symétrie par rapport à l'axe de l'ordonnée ");
        }
        else if(( p.getX() == q.getX() ) && (p.getY() == -q.getY() )) {
            System.out.println("symétrie par rapport à l'axe des abscisses ");
        }
        else if((p.getX() == -q.getX()) &&(p.getY() == -q.getY())) {
            System.out.println("symétrie par rapport au centre ");
        } else {
            System.out.println("Points qcq");
        }
    }
}

```

Exercice n°3

Question : Que fournit le programme Java suivant ?

1°) a.f(b);	Appel de la méthode f(byte b)
2°) a.f(n, q);	Ambiguïté d'appel : f (int n, double y) f (long n, float x)
3°) a.f(p);	Appel de la méthode f(int n), conversion implicit : short -> int
4°) a.f(q);	Appel de la méthode f(float x), conversion implicite : long -> float
5°) a.f(b+1);	Appel de la méthode f(int n), b+1 sera interprété comme un int
6°) a.f(b++);	Appel de la méthode f(byte b), b++ sera interprété comme un byte
7°) a.f(p, n);	Appel de la méthode f(int n, int m), conversion implicite : short -> int
8°) a.f(b, x);	Ambiguïté d'appel : f(long n, float x) f (int n, double y)
9°) a.f(n, y);	Appel de la méthode : f(int n, double y)
10°) a.f(p, q);	Ambiguïté d'appel f(int n, double y) f (long n, float x)

Exercice n°4

Question : Que fournit le programme Java suivant ?

```
class Inconnue {
    public Inconnue (int coeff){
        nbre *= coeff ;
        nbre += decal ;
    }
    public void affiche (){
        System.out.println ("nbre = " + nbre + " decal = " + decal);
    }
    private int nbre = 20 ;
    private int decal ;
}
public class Test{
    public static void main (String args[]){
        Inconnue a = new Inconnue (5);
        a.affiche();
    }
}
```

- ✓ Le programme commence par créer l'objet **a** en lui passant la valeur 5
- ✓ Pendant l'instanciation, les variables (attributs) d'instances vont être initialisés automatiquement : (nbre à 20 et decal à 0).
- ✓ Le constructeur va calculer l'attribut privé nbre
- ✓ Avec l'objet a on peut invoquer la méthode affiche pour afficher les champs privés : nbre = 10 decal = 0

Exercice n°5

```

package exercice5;

class Point {
    private double x;
    private double y;

    public Point(double x, double y) {
        this.x=x;
        this.y=y;
    }
    public double distance() {
        double dis;
        dis = Math.sqrt( (Math.pow(this.x, 2)) + Math.pow(this.y, 2));
        return dis;
    }
}

class Cercle {
    private String nom;
    private double r;

    public Cercle(String nom, double r) {
        this.nom = nom;
        this.r = r;
    }
    public double getR() {
        return this.r;
    }
}

import java.util.Scanner;

public class Principale {

    public static void main(String[] args) {

        Scanner sc = new Scanner (System.in);
        System.out.print("Donner la valeur de x : ");
        double valx = sc.nextDouble();
        System.out.print("Donner la valeur de y : ");
        double valy = sc.nextDouble();
        sc.nextLine(); //Vide le buffer : \n
        Point p = new Point(valx,valy);
        System.out.println("Coordonnées du point : " + valx + " - " + valy);
        System.out.print("Donner le nom du cercle : ");
        String nom = sc.nextLine();
        System.out.print("Donner le rayon du cercle : ");
        double r = sc.nextDouble();
        Cercle c = new Cercle(nom, r);
        System.out.println("Cercle : " + nom + " - " + r);
        if(p.distance() < c.getR()) {
            System.out.println(p.distance() + " ==> Point dans le cercle");
        } else {
            System.out.println(p.distance() + " ==> Point en dehors du
cercle");
        }
    }
}

```

Exercice n°6

```

package exercice6;

class Livre {
    private String nom;
    private String titre;
    private int nbPages;
    static private int nbLivres;
    private int numLivre;

    public Livre(String nom, String titre) {
        nbLivres++;
        this.nom = nom;
        this.titre = titre;
        this.numLivre += nbLivres;
    }

    public Livre(String nom, String titre, int nbpages) {
        nbLivres++;
        this.nom = nom;
        this.titre = titre;
        this.nbPages = nbpages;
        this.numLivre += nbLivres;
    }

    public String getNom() { return nom; }

    public void setNom(String nom) { this.nom = nom; }

    public String getTitre() { return titre; }

    public void setTitre(String titre) { this.titre = titre; }

    public int getNbpages() { return nbPages; }

    public void setNbpages(int nbpages) { this.nbPages = nbpages; }

    public int getNBLivres() { return this.nbLivres; }

    public int getNumLivre() { return numLivre; }
}

public class TestLivre {

    public static void main(String[] args) {
        Livre l1 = new Livre("Programmation Java", "abcd");
        Livre l2 = new Livre("Programmation PHP", "efgh", 500);
        System.out.println("Nombre des livres " + l1.getNBLivres());
        System.out.println("Nombre des livres " + l2.getNBLivres());

        System.out.println("Livre 1: " + l1.getNumLivre());
        System.out.println("Livre 2: " + l2.getNumLivre());
    }
}

```

Exercise 7

```
package exercice7;

class Point {
    private int x;
    private int y;
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public void affiche() {
        System.out.println("Coordonnées: " +x+ " - "+y);
    }
}

class PointPlus extends Point{
    private char nom;
    public PointPlus(int x, int y, char nom) {
        super(x,y);
        this.nom = nom;
    }
    //Redéfinition de la méthode de la classe de base
    public void affiche() {
        System.out.println("Point: " + this.nom);
        super.affiche(); //Appel de la méthode affiche de la classe de base
    }
}

public class TestPointPlus {
    public static void main(String[] args) {
        PointPlus pp = new PointPlus(2, 3, 'C');
        pp.affiche();
    }
}
```

Exercise 8

```

package exercice8;

class Employe {
    private String nom;
    private String prenom;
    private double salaire;
    public Employe(String nom, String prenom, double salaire){
        this.nom = nom;
        this.prenom = prenom;
        this.salaire = salaire;
    }
    public void affiche(){
        System.out.println(("Bonjour Mr : " + nom + " " + prenom + " Votre salaire : "
+ salaire));
    }
    public String getNom() { return nom; }
    public String getPrenom() { return prenom; }
    public double getSalaire() { return salaire; }
}

class Manager extends Employe {
    private String dept;
    public Manager(String nom, String prenom, double salaire, String dept){
        super(nom, prenom, salaire); //Appel du constructeur de la classe
                                   //de base pour initialiser les champs privés
        this.dept = dept;
    }
    public void affiche(){
        System.out.println("Bonjour Manager : " + super.getNom() + " " + super.getPrenom() + " - " + "depart : " + dept);
    }
}

class Adviser extends Employe {
    private String spe;
    public Adviser(String nom, String prenom, double salaire, String spe) {
        super(nom, prenom, salaire); //Appel du constructeur de la classe de base pour initialiser les champs privés
        this.spe = spe;
    }
    public void affiche(){
        System.out.println("Bonjour Adviser : " + super.getNom() + " - " + super.getPrenom() + " - " + "specialite : " + spe);
    }
}

public class Principale {
    public static void main(String[] args) {
        Employe liste[] = new Employe[4]; //Sauvegarder tous les objets créés dans un tableau d'Employe
        liste[0] = new Manager("Jhon", "Bisard", 2500, "Informatique");
        liste[1] = new Manager("Sara", "Lablanc", 2300, "Gestion");
        liste[2] = new Adviser("Wafa", "Artisto", 2800, "Auditeur");
        liste[3] = new Adviser("Liona", "Wing", 2100, "Consultant");
        for(int i =0; i<4; i++) {
            System.out.println("Employe n " + i);
            liste[i].affiche();
        }
    }
}

```

Exercise 9

```

package exercice9;

abstract class Animal {
    private String nom;
    static int nbAnimal;
    public Animal(String nom) {
        nbAnimal++;
        this.nom=nom;
    }
    public abstract void parler();
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public static int getNbAnimal() { return nbAnimal; }
}

class Chien extends Animal {
    public Chien(String nom) {
        super(nom);
    }
    public void parler() {
        System.out.print("Chien :: ");
    }
}

class Chat extends Animal {
    public Chat(String nom) {
        super(nom);
    }
    public void parler() {
        System.out.print("Chat :: ");
    }
}

public class TestAnimal {
    public static void main(String[] args) {
        Animal a[] = new Animal[4];
        a[0] = new Chien("Rex");
        a[1] = new Chat("Misty");
        a[2] = new Chien("Bella");
        a[3] = new Chat("Simba");

        int nbchat=0, nbchien=0;
        for(int i=0; i<a.length; i++) {
            a[i].parler();
            System.out.println(a[i].getNom());
            if(a[i] instanceof Chat) {
                nbchat++;
            } else if(a[i] instanceof Chien) {
                nbchien++;
            }
        }
        System.out.println("Nombre animaux = " + Animal.nbAnimal);
        System.out.println("Nombre de Chats: " + nbchat);
        System.out.println("Nombre de Chiens: " + nbchien);
    }
}

```

Exercice 10

```

package exercice10;

interface Affichable {
    void affiche() ;
}
class Entier implements Affichable {
    private int val ;
    public Entier (int n) {
        this.val=n ;
    }
    // définition de la méthode abstraite
    public void affiche () {
        System.out.println ("Je suis un entier de valeur " + val) ;
    }
}
class Flottant implements Affichable {
    private float val ;
    public Flottant (float n) {
        this.val=n ;
    }
    // définition de la méthode abstraite
    public void affiche () {
        System.out.println (" Je suis un flottant de valeur " + val) ;
    }
}
public class TestInterfacePoly {
    public static void main (String [] args){
        Affichable [] tab ;
        /* Tab est un tableau de références d'objets de classes
           différentes, toutes implémentant l'interface Affichable*/
        tab = new Affichable[5];
        tab[0] = new Entier (25) ;
        tab[1] = new Flottant (1.25F) ;
        tab[2] = new Entier(17) ;
        tab[3] = new Flottant(3.14F);
        tab[4] = new Entier(36);
        int nbe=0, nbf=0;
        for (int i=0; i < tab.length; ++i){
            tab[i].affiche();
            if(tab[i] instanceof Entier) {
                nbe++; }
            else if(tab[i] instanceof Flottant) {
                nbf++; }
        }
        System.out.println("Objet entier: "+nbe);
        System.out.println("Objet flottant: "+nbf);
    }
}

```